Informe de operación — ComicRent P10

Simulacro del 12 de octubre de 2026. Todas las horas están en UTC. La evidencia del canary y del diagnóstico incluye el AnalysisRun, el registro JSON, los spans y las marcas de tiempo originales.

1. SLO declarados

Disponibilidad: al menos 99.9% de las solicitudes de usuario al `api-gateway` sin `5xx` ni timeout en 30 días móviles. El presupuesto es 0.1% de solicitudes fallidas. Es un compromiso defendible para una plataforma académica con varias dependencias; 99.99% permitiría diez veces menos errores sin que exista evidencia de capacidad operativa para sostenerlo. **Latencia:** al menos 95% de las solicitudes terminan en 500 ms o menos en la misma ventana; el presupuesto es 5% de solicitudes lentas. El límite responde a la experiencia interactiva y coincide con el control del canary. Se excluyen `/health` y `/metrics`. Los SLI y presupuestos se calculan con contadores e histogramas HTTP, no con disponibilidad de Pods.

2. Fallo inducido

El commit de prueba GitOps `7540a4b` apuntó la autenticación de la revisión **candidata** a `auth-backend.p10.invalid`. Se enviaron 80 peticiones de login con datos ficticios directamente al Pod candidato; las 80 devolvieron `502`. El hash estable `bf669688c` conservó sus tres réplicas y no recibió esas peticiones. El fallo fue de resolución DNS de una dependencia, distinto al incidente de P8.

3. Tiempo de detección

La primera petición fallida quedó registrada a las **23:01:38.176**. `ComicRentAvailabilityBudgetFastBurn` entró en `pending` a las **23:03:22.384**: primera señal de alerta documentada **1 min 44.208 s** después. Con `for: 2m`, pudo pasar a `firing` desde las **23:05:22.384**. Esta última hora es un límite calculado a partir de la regla, no una marca de transición observada.

4. Recorrido hasta la causa

Desde la alerta `firing` se consultó en Prometheus el error por ruta: `/api/auth/login` concentraba los `5xx`. En Loki, el `correlation_id` `p10-block7-fault-20261012T230138Z-1` llevó al registro JSON `http.request.failed`, HTTP `502`, con `trace_id` `a3f84aa1985dc03f443808ff9036e098`. Ese identificador abrió en Tempo la traza cuyo span saliente apuntaba a `auth-backend.p10.invalid` y terminaba en `ENOTFOUND`. La consulta conjunta tomó **4.6 s**. Desde el primer instante posible de `firing` hasta el fin del diagnóstico a las **23:06:39.310** pasaron, como máximo, **1 min 16.93 s**, por debajo de tres minutos; no se registró la hora exacta en que el operador vio por primera vez la alerta.

5. Tiempo de recuperación

El AnalysisRun fallido concluyó a las **23:04:29** con tasa de error candidata `1.0`, superior al límite `0.001`. En esa misma marca se observó `RolloutAborted`: Argo Rollouts detuvo la promoción y devolvió el selector canary al hash estable `bf669688c`, **2 min 50.824 s** desde la primera petición fallida. El servicio estable nunca salió de operación. El revert GitOps `60a2cc0` reparó la configuración declarada; el estado del Rollout volvió a `Healthy/Completed` a las **23:07:48**, **6 min 9.824 s** desde el inicio del fallo.

6. Alerta faltante o sobrante

La alerta rápida se activó por peticiones sintéticas enviadas directamente al Pod candidato, aunque el servicio estable siguió sano: fue ruido operativo. Después se separó `canary-test` de `user`, se hizo que el Service público alcanzara ambas revisiones y se limitó el AnalysisTemplate a métricas `user`. Una segunda prueba por el Service público confirmó cuatro promociones sanas y otro aborto automático por `502` del candidato, con la alerta de presupuesto disparada. La mejora es posterior al primer simulacro y no altera retrospectivamente sus tiempos.